# Python Interview Knowledge

Beginner-friendly, job-ready reference for Python fresher interviews

Purpose
Core Python language, practical coding, OOP, errors, environments and performance.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

# Table of Contents

1. 1. Python mental model and data types
3. 2. Collections and common operations
4. 3. Functions, arguments and scope
5. 4. OOP, inheritance and polymorphism
7. 5. Exceptions and file handling
8. 6. Modules, packages and environments
9. 7. Iterators, generators, decorators and comprehensions
10. 8. Complexity and coding patterns
11. 9. Troubleshooting and project use cases

# 1. Python mental model and data types

Python is dynamically typed: names refer to objects and the runtime object has a type. Type annotations improve tooling and readability but do not by themselves enforce runtime types.

Important built-ins include int, float, bool, str, list, tuple, set, dict, bytes and None. `None` represents absence of a value and should not be confused with 0, False or an empty container.

Equality and identity are different. `==` compares values according to an object's equality rules; `is` tests object identity. The standard identity check `x is None` is appropriate; `x is 5` is not a value-comparison pattern.

- Dynamic typing does not mean Python is untyped; objects still have types.
- Names can be rebound to objects of different types.
- Use annotations such as `def score(x: float) -> float` to communicate contracts.

| Concept | Meaning | Example |
|---|---|---|
| Type | What operations an object supports | str, list, dict |
| Identity | Which object a name refers to | a is b |
| Equality | Whether values compare equal | a == b |
| None | No value / missing result | if result is None: |

## Mutability

Mutable objects can change after creation; immutable objects cannot. Lists, dicts and sets are mutable. Strings, numbers and tuples are immutable. A tuple can contain a mutable list, so tuple immutability does not make every nested object immutable.

- Aliasing is the key pitfall: two names can point to the same mutable object.
- A shallow copy duplicates the outer container but can share nested objects.
- `copy.deepcopy` is not automatically the best answer; explicit reconstruction can be clearer and cheaper.

# 2. Collections and common operations

Choose a collection from the operations you need. Lists preserve order and support indexing. Tuples are immutable sequences. Sets store unique hashable elements. Dictionaries map hashable keys to values and preserve insertion order in modern Python.

Average-case dictionary/set membership is commonly described as O(1), while list membership is O(n). These are typical complexity expectations, not guarantees that every operation has a fixed runtime.

| Structure | Mutable | Typical membership | Best fit |
|---|---|---|---|
| list | Yes | O(n) | Ordered sequence |
| tuple | No | O(n) | Fixed sequence/record |
| set | Yes | O(1) average | Uniqueness/membership |
| dict | Yes | O(1) average by key | Key-value lookup |

```
scores = {"Asha": 82, "Ravi": 91}
scores["Mina"] = 88
scores["Ravi"] += 3

unique_ids = set([10, 10, 12, 13])
print(unique_ids)
```

# 3. Functions, arguments and scope

Functions are first-class objects: they can be stored, passed and returned. `*args` collects extra positional arguments and `**kwargs` collects extra keyword arguments. In calls, `*` and `**` perform unpacking.

Python name lookup follows LEGB: Local, Enclosing, Global, Built-in. `nonlocal` changes an enclosing binding; `global` changes a module-level binding. Use shared state sparingly because it complicates tests.

Lambda functions are useful for short expressions such as sort keys; a normal `def` is clearer for multi-step logic.

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

people = [{"name":"Asha", "score":92}, {"name":"Ravi", "score":70}]
people.sort(key=lambda p: p["score"], reverse=True)
```

# 4. OOP, inheritance and polymorphism

A class defines behavior and state; an object is an instance. Encapsulation is about controlling access to state and exposing useful operations. Inheritance specializes a type; composition means one object uses another. Polymorphism lets code depend on a common behavior instead of a concrete class.

Python uses conventions for internal attributes. A leading underscore communicates internal use; double leading underscores trigger name mangling, which is not a security boundary.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

class SavingsAccount(BankAccount):
    def add_interest(self, rate):
        self._balance *= 1 + rate
```

## Composition versus inheritance

If a service needs a repository, composition is often clearer: `ReportService(repository)`. Inheritance is appropriate when the subtype truly satisfies the parent abstraction and the substitution remains valid.

- Prefer composition when behavior can be swapped independently.
- Use inheritance when the relationship is genuinely an “is-a” relationship.

# 5. Exceptions and file handling

Catch the narrowest exception you can handle. `finally` is useful for cleanup, while context managers (`with`) are the standard way to manage files and many other resources.

Do not use `except Exception` around an entire application layer just to keep it running. It can hide programming bugs. Catch expected operational failures and convert them into useful domain/API errors where appropriate.

```
from pathlib import Path

try:
    with Path("notes.txt").open(encoding="utf-8") as f:
        content = f.read()
except FileNotFoundError:
    content = ""
except OSError as exc:
    raise RuntimeError("Could not read notes") from exc
```

# 6. Modules, packages and environments

A module is a Python file that can be imported. A package groups modules. A virtual environment isolates project dependencies from system Python. A requirements file records installable dependencies so another environment can reproduce the project setup.

Dependency reproducibility is a trade-off: exact pins maximize repeatability; compatible ranges can make maintenance easier. For interview answers, say which goal you are optimizing.

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
python -m pip install fastapi
python -m pip freeze > requirements.txt
```

# 7. Iterators, generators, decorators and comprehensions

An iterable can provide an iterator. An iterator implements the iteration protocol and yields one item at a time. A generator function uses `yield`, making lazy pipelines possible without storing every result in memory.

A decorator wraps or replaces a callable. `functools.wraps` preserves useful metadata from the wrapped function. Comprehensions are concise but should remain readable.

```
from functools import wraps

def logged(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@logged
def square(x):
    return x * x

values = (x*x for x in range(10))
```

# 8. Complexity and coding patterns

Big-O describes growth as input size changes. It is not a direct measurement of seconds. Interviewers expect you to identify avoidable repeated work and choose data structures accordingly.

Common patterns include dictionaries/sets for membership, two pointers for sorted arrays, sliding windows for contiguous ranges, stacks for nested structure, queues for breadth-first processing and binary search for sorted data.

| Operation | Typical complexity | Reason |
|---|---|---|
| List append | O(1) amortized | Dynamic array append |
| List front insert | O(n) | Elements shift |
| Dict lookup | O(1) average | Hash table |
| Sorting | O(n log n) typical | Timsort |
| Binary search | O(log n) | Halves search range |

```
def first_duplicate(values):
    seen = set()
    for value in values:
        if value in seen:
            return value
        seen.add(value)
    return None
```

# 9. Troubleshooting and project use cases

In a software project, Python often sits at the API, data-processing, automation and ML-integration layers. Troubleshooting should start by identifying the boundary: input validation, Python logic, database I/O, network I/O, serialization or external service behavior.

A slow endpoint that fetches 100,000 rows should not be “fixed” only by micro-optimizing a list comprehension. Push filtering/aggregation to the database, paginate and measure before optimizing Python code.

A memory-heavy CSV script should stream rows or batches rather than loading the entire file. Generators help only if downstream code also avoids retaining all values.

- A common bug is a mutable default argument such as `def add(x, bucket=[]):`. Use `None` and allocate inside.
- Do not mutate a collection while iterating unless the algorithm is designed for it.
- Avoid swallowing exceptions that would reveal a broken invariant.

## Interview question

**Why is a tuple immutable but it can contain a list?**

Strong answer: The tuple’s own sequence of references cannot be changed. The list object referenced by one element can still be mutated. Immutability is about the tuple object, not recursive immutability of everything reachable from it.

Follow-up: How does this affect hashing?

## Interview question

**Why should `is` not replace `==`?**

Strong answer: `is` tests identity, while `==` tests equality. Implementation details such as object interning can make identity appear to work for some values, but that is not a value-comparison contract.

Follow-up: Why is `x is None` recommended?

## Interview question

**What is duck typing?**

Strong answer: Code relies on the operations an object supports rather than requiring a specific concrete class. A service can accept any repository that provides the methods it needs. This enables simple polymorphism and test doubles.

Follow-up: How would you make the contract explicit with typing?

## Interview question

**Shallow copy versus deep copy?**

Strong answer: A shallow copy creates a new outer object but can retain references to nested mutable objects. A deep copy recursively copies supported nested objects. Use the smallest copying strategy that matches the ownership requirements.

Follow-up: Why can deep copy be expensive or surprising?

## Scenario: repeated database membership checks

Situation: A loop performs `if id in ids_list` thousands of times.

Reasoning: If order is not required, convert the IDs to a set once and use membership checks. If the data is already in the database, consider doing the operation in SQL rather than transferring unnecessary rows.

## Scenario: generator did not reduce memory

Situation: A developer changed a list comprehension to a generator, but memory use remains high.

Reasoning: Inspect downstream code. If the generator is immediately converted to a list or every yielded object is retained elsewhere, laziness does not reduce the final retained data.

Avoid: Changing syntax without changing retention.

## Interview question

**How would you explain Python to an interviewer in one minute?**

Strong answer: Python is a dynamically typed, high-level language with a large standard library and ecosystem. I use its built-in collections and functions for clear application code, virtual environments for dependency isolation, exceptions/context managers for reliable resource handling, and profiling plus appropriate data structures when performance matters.

Follow-up: What are Python’s main performance limitations?

## Revision checklist

- Explain identity versus equality.
- Choose list/tuple/set/dict from required operations.
- Explain mutable defaults and aliasing.
- Explain LEGB, closures and argument unpacking.
- Explain inheritance, composition and polymorphism.
- Use context managers and narrow exception handling.
- Explain virtual environments and dependency files.
- Differentiate iterable, iterator and generator.
- Explain decorators and `wraps`.
- Estimate time and space complexity.

# Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://docs.python.org/3/

https://packaging.python.org/

End of knowledge base document